

DEEP LEARNING AND SEQUENTIAL MODELS FOR BIG DATA ANALYTICS

Lab 1: Neural Networks (Multi-Layer Perceptron) for Classification and Regression

Title:

- (a) Building a Neural Network as a Classifier to implement Logic Gates including XOR**
 - (b) Building a Neural Network as a Regressor for USA House Price Prediction**
-

(a) Building a Neural Network as a Classifier for Logic Gates Including XOR

Learning Objectives:

- Understand how neural networks generalize perceptrons for complex logic (non-linear classification boundaries).
- Implement logic gates (AND, OR, XOR) using a feedforward neural network (multi-layer perceptron).
- Implement neural network as a classifier and understand the role of number of neurons in achieving accuracy.

Instructions & Code

Step 1: Data Setup for Logic Gates

```
import numpy as np
```

```
# Dataset for logic gates
```

```
X = np.array([
```

```
    [0, 0],
```

```
    [0, 1],
```

```
    [1, 0],
```

```
    [1, 1]
```

```
])
```

```
# Choose gate: Uncomment as needed
```

```
y_and = np.array([0, 0, 0, 1]) # AND gate
```

```
y_or = np.array([0, 1, 1, 1]) # OR gate
```

```
y_xor = np.array([0, 1, 1, 0]) # XOR gate
```

Step 2: Build and Train a Neural Network Classifier

Here, we use scikit-learn's MLPClassifier for simplicity:

python

```
from sklearn.neural_network import MLPClassifier
```

```
# Try with different logic gates; start with XOR
```

```
clf = MLPClassifier(hidden_layer_sizes=(4,), activation='relu', max_iter=5000,  
random_state=1)
```

```
clf.fit(X, y_xor)
```

```
# Test predictions
```

```
print("Input:\n", X)
```

```
print("Predicted Output:", clf.predict(X))
```

```
print("Actual Output: ", y_xor)
```

Step 3: Try Out AND and OR Gates (repeat for y_and or y_or above)

Change y_xor to y_and or y_or in the .fit() and see results.

Discussion:

- The XOR gate is not linearly separable and can't be learned with just a perceptron; the neural network's hidden layer is what makes this possible!
 - Try visualizing the decision boundary if desired or print. score(X, y_xor) for model accuracy.
-

(b) Building a Neural Network Regressor for USA House Price Prediction

Learning Objectives:

- Apply a neural network for real-world regression on tabular data.
- Use data normalization, splitting, and model evaluation.

Instructions & Code

Step 1: Load an Example Housing Dataset

We'll use the classic California housing dataset available in scikit-learn, similar to USA housing data.

python

```
# from sklearn.datasets import fetch_california_housing
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
import numpy as np
```

```
import pandas as pd
```

```
# Load dataset
```

```
# data = fetch_california_housing()
```

```
data = pd.read_csv("USA_Housing.csv")
```

```
data. Head
```

```
data.info
```

```
data.describe
```

```
data.columns
```

```
X = data[['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms',  
         'Avg. Area Number of Bedrooms', 'Area Population']]
```

```
X.shape
```

```
y = data[['Price']]
```

y.shape

X = data.data

y = data.target

Train/test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Standardize data

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

Step 2: Build and Train the Neural Network Regressor

python

from sklearn.neural_network **import** MLPRegressor

reg = MLPRegressor(hidden_layer_sizes=(64, 32), activation='relu', max_iter=1000,
random_state=1)

reg.fit(X_train, y_train)

regressor.score(X_test, y_test)

Predict and Evaluate

from sklearn.metrics **import** mean_absolute_error, mean_squared_error, r2_score

y_pred = reg.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

```
r2 = r2_score(y_test, y_pred)
```

```
print("Test Mean Absolute Error:", mae)
```

```
print("Test Mean Squared Error:", mse)
```

```
print("Test R^2 Score:", r2)
```

Step 3: Experiment

- Change the network size, activation, or regularization.
 - Try plotting predicted vs. actual prices for deeper understanding (optional).
-

Submission

- For (a): Output for all three logic gates (AND, OR, XOR) and network architecture used.
 - For (b): Training and test scores, model structure, and (optional) plot of predicted vs. actual prices.
-

Reflection Questions:

1. Why does the neural network successfully learn XOR but a single-layer perceptron cannot?
2. What is the impact of hidden layer size on logic gate classification?
3. What challenges do you notice in house price regression compared to classification?
4. How does normalization affect the regression model's performance?